

Containers

Den kanske viktigaste delen i STL är nog de olika *containers* som finns implementerade. Dessa containers fyller de flesta programs behov av klasser för att spara data i. En mängd olika containers med olika användningsområden finns. Dessa kan indelas i två grupper beroende på hur data i dem kan accesseras.

Sekvenscontainers

Dessa containers har fått sitt namn av att data är lagrat i strikt sekventiell ordning. Man kan accessera all data i en slumpmässig ordning. De olika containers som finns är:

- `vector` som fungerar som en normal vektor, men den är mera dynamisk och kan ändra storlek om det behövs. Normala vektorer har en fast storlek. Access av data är snabbt, d.v.s. i konstant tid. Använd dessa istället för normala vektorer om storleken kan komma att ändra under exekveringen.
- `deque` är en implementation av en normal kö, där data kan effektivt sättas in och raderas i början och slutet. Kan användas då man behöver t.ex. en FIFO-kö för att organisera data för processering.
- `list` är en vanlig lista av element. Access till data i mitten av listan sker i linjär tid. Kan användas där man behöver en lista med element och inte behöver göra alltför frekventa accesser till data i mitten av listan. Insättningar och raderingar i mitten av en lista är dock effektiva. En lista kan inte accesseras i slumpmässig ordning.

Sorterade associativa containers

Associativa containers skiljer sig från normala containers i och med att man i här använder sig av en *nyckel* för att accessera data. Ett naturligt exempel kan vara att lagra personer i ett register på bas av t.ex. en medlemsnummer eller deras socialskyddssignum. All data sorteras på bas av nyckeln. Om man använder en nyckel som inte kan (eller bör) direkt jämföras med `==` kan man till konstruktorn ge med en egen jämförelsefunktion. De associativa containers som kan användas är:

- `set` som definierar en enkel klass för hantering av en mängd av data. Denna är sorterad på bas av en nyckel.
- `map` fungerar på samma sätt som `set`, men innehåller endel extra metoder som gör den lättare att använda och den klass som normalt används.
- `multiset` är ett `set` som tillåter multipla element att ha samma nyckel.
- `multimap` är en `map` som tillåter multipla element att ha samma nyckel.

Instättning och access av data

Att sätta in data i sekvenscontainers är enkelt, men varierar lite beroende på vilken container som

används. Som exempel visas hur data läses in från tangentbordet kan placeras i en `vector` ända tills strängen `end` läses in:

```
#include <vector>
#include <string>

int main (int argc, char * argv[]) {
    vector<string> Data;
    string Value;

    cout << "Skriv text och avsluta med 'end':" << endl;

    // läs data från tangentbordet
    while ( cin >> Value ) {
        // är vi klara med insättngen?
        if ( Value == "end" ) {
            // jep, hoppa ut
            break;
        }

        // addera till vektorn
        Data.push_back ( Value );
    }

    // iterera över alla värden
    for ( unsigned int Index = 0; Index < Data.size (); Index++ ) {
        cout << Index << ": " << Data[Index] << endl;
    }
}
```

Programmet visar hur man med hjälp av metoden `push_back` placerar data i slutet på vektorn.

Vi kunde byta ut `vector` i exemplet ovan till `deque` och få exakt samma resultat, eftersom båda har samma metoder definierade, och båda kan accesseras slumpmässigt via `operator[]`.

Exempel på användning av en associativ container (`map`) finns nedan. Vi skapar här ett program som räknar förekomsten av ord i en fil, och skriver sedan ut en frekvenstabell. som nyckel används själva ordet, medan de egentliga datat för varje element är en `int` som anger antalet förekomster. Orden är automatiskt sorterade. I en `map` kan man sätta in element med `operator[]`. Om det redan finns ett element med samma nyckel skrivs det över. Om man vill tillåta flera element med samma nyckel kan man använda `multimap` och `multiset`.

```
#include <map>
#include <string>
#include <fstream>
#include <stdlib.h>

int main (int argc, char * argv[]) {
    // definiera en 'map' av string->int
    map<string, int> Text;
    string Value;

    // har vi parametrar?
    if ( argc != 2 ) {
        cout << "Fel antal parametrar!" << endl;
        cout << "Användning: " << argv[0] << " filnamn" << endl;
        exit ( EXIT_FAILURE );
    }

    // öppna en fil
```

```

ifstream Data ( argv[1] );
if ( ! Data ) {
    // kunde inte öppna filen
    cout << "Kunde inte öppna filen: " << argv[1] << endl;
    exit ( EXIT_FAILURE );
}

// iterera över all ord i filen
while ( Data >> Value ) {
    // lagra i vår map
    Text[Value]++;
}

// iterera över våra ord och skriv ut deras frekvenser
map<string, int>::iterator It = Text.begin ();
for ( ; It != Text.end (); It++ ) {
    // en 'map' innehåller 'pair':s som innehåller vår data
    pair<string, int> P = *It;
    cout << P.second << "\\t" << P.first << endl;
}

cout << endl << "Totalt " << Text.size () << " unika ord." << endl;

// close stream
Data.close ();
}

```

Exemplet ovan visar även filhantering med hjälp av klassen `ifstream`.

De flesta klasser har metoder för att placera data först och sist i containern med hjälp av metoderna `push_front` och `push_back`. Vissa kan även använda `operator[]`. Man kunde även använda metoden `insert()` som använder sig av en *iterator* (se [avsnittet Iterationer](#)) för att berätta var i containern det nya elementet skall placeras.

Radering av data

De flesta containers har metoder för att radera data. Man kan i de flesta fallen använda både `pop_front()` och `pop_back()` för att radera ett element. Denna metod kommer dock inte att frigöra minne som elementet eventuellt allokerat, utan man måste manuellt förstöra elementet med t.ex. `delete`. Exemplet nedan visar hur man kan göra:

```

#include <map>
#include <string>
#include <iostream>

int main (int argc, char * argv[]) {
    map<int, string *> PhoneBook;
    string Value;

    // sätt in element i vår telefonbok
    PhoneBook[5551234] = new string ("Foo Bar");
    PhoneBook[5555678] = new string ("Joe Q. Public");
    PhoneBook[5553456] = new string ("John Smith");
    PhoneBook[5559876] = new string ("Hugo Hacker");

    cout << "Vi har " << PhoneBook.size () << " personer." << endl;

    // vi vill radera ett element från vår telefonbok
    string * DeleteMe = PhoneBook[5553456];
}

```

```

delete DeleteMe;

// utför raderingen
PhoneBook.erase ( PhoneBook.find (5553456));

cout << "Vi har nu " << PhoneBook.size () << " personer." << endl;
}

```

Detta program opererar på data som är `string *` och som allokeras då det sätts in. Om vi när vi raderar ett element från listan inte även frpgör minnet kommer vårt program att läcka minne. Vi accesserar elementet med `operator[]` och raderar det, och sedan raderar vi själva elementet i containern med `erase()`. `Erase` tar som parameter en `iterator` som visar vilket element som skall raderas, och det elementet hittar vi med t.ex. `find()`. Vi kunde alternativt skrivit:

```

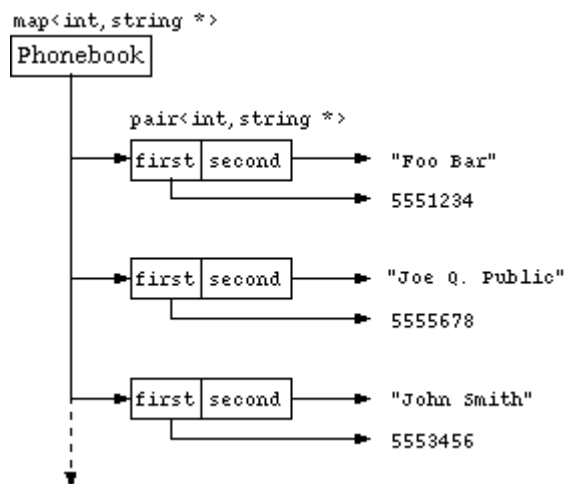
// vi vill radera ett element från vår telefonbok
map<int, string *>::iterator It = PhoneBook.find ( 5553456 );
delete (*It).second;

// utför raderingen
PhoneBook.erase ( It );

```

Denna version är effektivare, eftersom vi nu utför endast en sökning. Som vi redan såg i ett tidigare exempel användes en klass `pair` för att accessera det data som finns i ett element. I en associativ container är varje element egentligen en instans av `pair`, som är templatiserad att använda samma argument som containern. En instans av `pair` innehåller två objekt, det första är nyckeln (accessera via medlemmen `first`) och det andra själva data (accessera via `second`). I vårt fall ovan kan man avreferera `It` för att få den `pair` som innehåller uppgifterna. Vår `map` ser ut som bilden nedan illustrerar:

Figur 24-1. Uppbyggnaden av en map



Det kan verka svårt till en början att förstå alla avrefereringar och returnvärden rätt, men efter en tids användande tycker man vanligen att det sätt på vilket STL fungerar är naturligt.

Övrig funktionalitet hos containers

Det finns även annan funktionalitet hos containers. Den mest använda är troligtvis att få ut antalet element som en container innehåller. Detta kan göras såsom exemplen ovan visar med hjälp av metoden `size()`. Man kan i de flesta fall jämföra om två containers innehåller samma element genom den överlagrade operatoren `==`. Det är även enkelt att kopiera containers via en konstruktor

som tar som parameter en existerande container. Detta fungerar dock endast då både originalet och den blivande kopian är av samma typ.

En viktig och ofta använd metod är `find()` som försöker hitta ett element i en associativ container på bas av en nyckel. Dett är den normala metoden för att söka element.

[Förutgående](#)
Standard Template Library

[Hem](#)
[Upp](#)

[Nästa](#)
Iteratorer